

VITyarthi – Build Your Own Project

SMART IMAGE EDGE & SHAPE ANALYZER

A Command-Line Computer Vision Project Using OpenCV

Course Domain: Computer Vision

Project Type: Classical Image Processing / Feature Extraction

Submitted as a course project demonstrating practical application of image preprocessing, edge detection, Hough Transform, contour analysis, and corner detection.

1. Introduction

Computer Vision enables computers to extract useful information from digital images. This project implements a compact command-line Computer Vision pipeline that identifies structural features in an input image. Instead of relying on a black-box deep-learning model, the system demonstrates fundamental image-processing algorithms that are directly connected to Computer Vision coursework.

The system first preprocesses an image, then performs four major analysis operations: Canny edge detection, Hough line detection, contour analysis, and Harris corner detection. The results are saved as annotated images and a machine-readable JSON report.

The project is designed for reproducibility. A built-in synthetic sample generator is included, so an evaluator can run the complete project without downloading a dataset.

2. Problem Statement

Images contain important geometric information such as object boundaries, straight lines, shapes, and corners. Manually identifying these features is time-consuming and may be inconsistent. The problem addressed by this project is to design a lightweight automated system that accepts an image and extracts these useful structural features through standard Computer Vision techniques.

Proposed solution: Build a modular command-line analyzer that preprocesses an image, extracts edges, detects prominent straight lines, analyzes contours, detects corner-like points, and produces visual and numerical outputs.

3. Functional Requirements

ID	Functional Requirement	Input	Output
FR1	Load and validate image	Image path	Validated image
FR2	Preprocess image	Color image	Grayscale + blurred image
FR3	Detect edges	Blurred image	Canny edge map
FR4	Detect straight lines	Edge map	Line overlay + count
FR5	Analyze contours	Edge map	Contours, area, perimeter, boxes
FR6	Detect corners	Grayscale image	Corner overlay + count
FR7	Generate report	All results	PNG files + JSON summary

Major modules:

- Preprocessing module – image loading, grayscale conversion, and Gaussian smoothing.
- Edge Detection module – Canny operator for extracting boundaries.
- Line Detection module – probabilistic Hough Transform for prominent straight lines.
- Contour Analysis module – contour extraction and basic geometric measurements.
- Corner Detection module – Harris corner response and point visualization.
- Reporting module – saves outputs and JSON statistics.

4. Non-Functional Requirements

Requirement	Project implementation
Performance	Processes normal academic-size images in a short command-line execution.
Usability	Simple CLI arguments and clear README instructions.
Reliability	Invalid image paths and invalid thresholds are rejected with meaningful errors.
Maintainability	Each Computer Vision operation is separated into a dedicated Python module.

Requirement	Project implementation
Scalability	The pipeline can be extended to batch processing and additional detectors.
Resource Efficiency	No database or external service is required; processing is performed locally.
Testability	Automated unit tests validate core processing functions.

5. System Architecture

The architecture follows a sequential modular pipeline. The CLI receives the image path and processing parameters. The image is passed to preprocessing and then to independent analysis modules. Their results are collected by the reporting module.

```

+-----+
| Command Line User |
+-----+
|
v
+-----+
| CLI / Arguments |
+-----+
|
v
+-----+
| Preprocessing |
| Gray + Gaussian |
+-----+
| | |
v v v
Canny Hough Contours
| | |
+-----+-----> Measurements / Visual Results
|
v
+-----+
| Harris Corners |
+-----+
|
v
+-----+
| Reporter |
| PNG + JSON |
+-----+

```

6. Design Diagrams

6.1 Use Case Diagram

```
+-----+
| Smart Image Edge & Shape Analyzer |
| |
User --->| (Load Image) |
| | |
| (Preprocess Image) |
| | |
| +---+---+---+---+ |
| | | | | |
| v v v v |
|Edges Lines Contours Corners |
| | |
| (Generate Report) |
+-----+
```

6.2 Workflow Diagram

```
Start -> Read Image -> Validate -> Grayscale -> Gaussian Blur
|
+-----+-----+
| | | |
Canny Hough Contours Harris
| | | |
Edge Map Lines Area/Perimeter/Box Corners
+-----+-----+
|
Save PNG + JSON
|
End
```

6.3 Sequence Diagram

```
User -> CLI: run command
CLI -> Preprocessor: load/preprocess image
Preprocessor -> CLI: image + grayscale
CLI -> Canny: detect edges
Canny -> CLI: edge map
CLI -> Hough: detect lines
Hough -> CLI: line result
CLI -> Contours: analyze shapes
Contours -> CLI: measurements
CLI -> Harris: detect corners
Harris -> CLI: corner result
CLI -> Reporter: save results
Reporter -> User: PNG files + JSON report
```

6.4 Component / Class Diagram

```
main.py
|-- cli.py
|-- preprocessing.py
|-- edge_detection.py
|-- line_detection.py
|-- contour_analysis.py
|-- corner_detection.py
|-- reporter.py
`-- generate_sample.py
```

6.5 Storage Design

No database is required. The system uses file-based storage. The input image is read from the local filesystem and the results are stored as PNG images and an analysis.json file. This is appropriate because the project is an image-analysis utility rather than a transactional information system.

7. Design Decisions & Rationale

- Classical Computer Vision was selected so the project visibly demonstrates syllabus-level image-processing algorithms.
- Canny was selected because it provides a standard approach for detecting strong image boundaries.
- HoughLinesP was selected because it detects finite line segments and produces an easily visualized result.
- Contours were selected for extracting object boundaries and simple geometric measurements.
- Harris Corner Detection was selected to demonstrate local feature detection at points with strong intensity changes.
- CLI execution was selected to satisfy terminal-based evaluation and improve reproducibility.
- A synthetic sample generator avoids dependency on an external dataset and makes testing deterministic.

8. Implementation Details

The project is implemented in Python using OpenCV and NumPy. The code is divided into small modules rather than placing the entire pipeline in one file.

Preprocessing: `cv2.imread` loads the image, `cv2.cvtColor` converts it to grayscale, and `cv2.GaussianBlur` reduces small-scale noise.

Edge detection: `cv2.Canny` generates a binary edge map using configurable lower and upper thresholds.

Line detection: `cv2.HoughLinesP` is applied to the edge map. Detected line segments are drawn over the original image.

Contour analysis: `cv2.findContours` extracts external contours. For contours above a minimum area, the project calculates area, perimeter, and a bounding rectangle.

Corner detection: `cv2.cornerHarris` calculates a corner response map. Strong responses are visualized as points.

Reporting: OpenCV writes four PNG result files and Python's `json` module writes a structured analysis report.

Command-line example: `python -m src.main --input data/sample_scene.png --output outputs`

9. Dataset, Method Selection & Evaluation

Dataset description: The repository contains a generated synthetic image with rectangles, a circle, line segments, and a polygon. This image acts as a reproducible test input. The application also accepts external JPG/PNG images supplied by the evaluator.

Model selection rationale: This is not a machine-learning classification project. It uses deterministic classical Computer Vision algorithms because the objective is feature extraction rather than learned prediction.

Evaluation methodology: The implementation is evaluated using automated unit tests, successful end-to-end execution, output-file generation, and visual inspection of the generated feature overlays. The sample run detected 17 line segments, 6 contours above the configured area threshold, and 80 displayed Harris corner points.

Important interpretation: Detection counts depend on image content and threshold parameters; they are not accuracy scores.

10. Screenshots / Results

Sample image size: 800 × 500 pixels. Default parameters: Canny 50/150, Hough threshold 80.

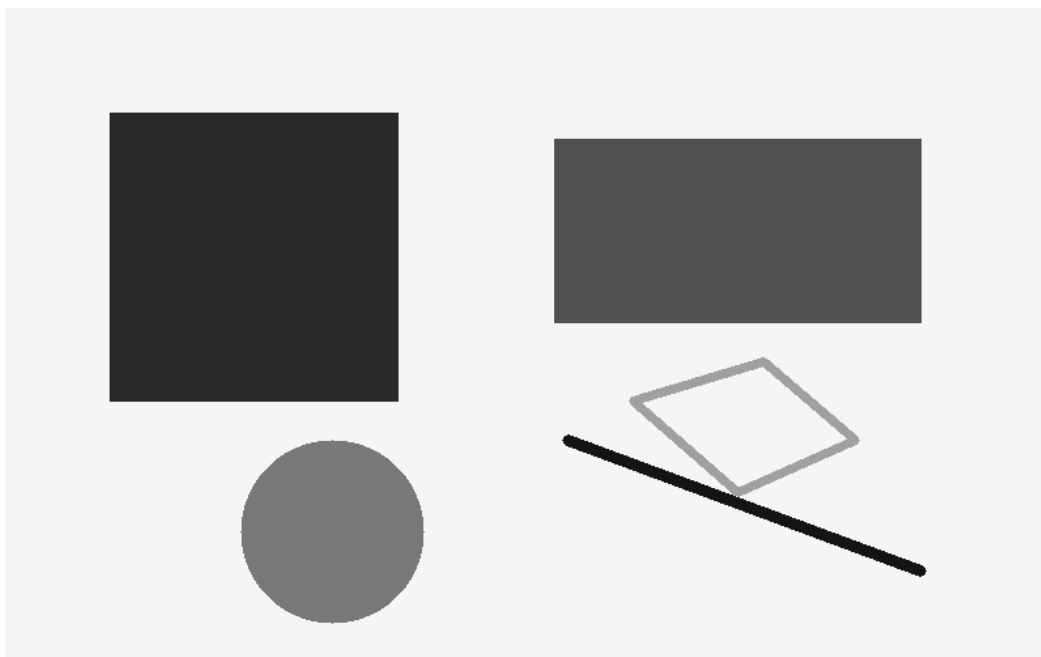


Figure 1. Generated sample input image.

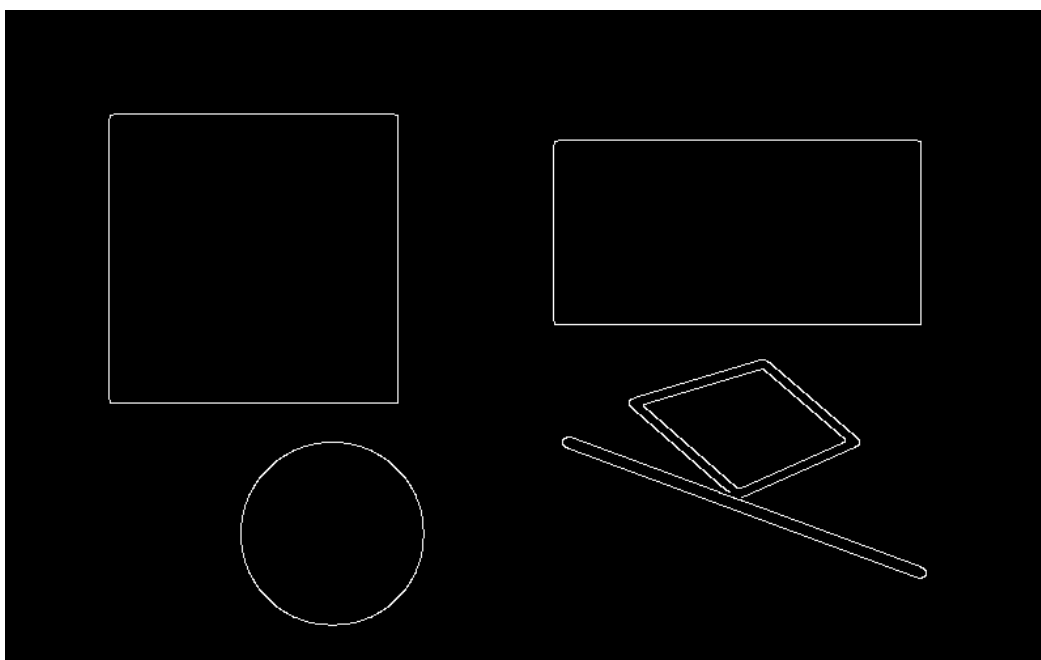


Figure 2. Canny edge map.

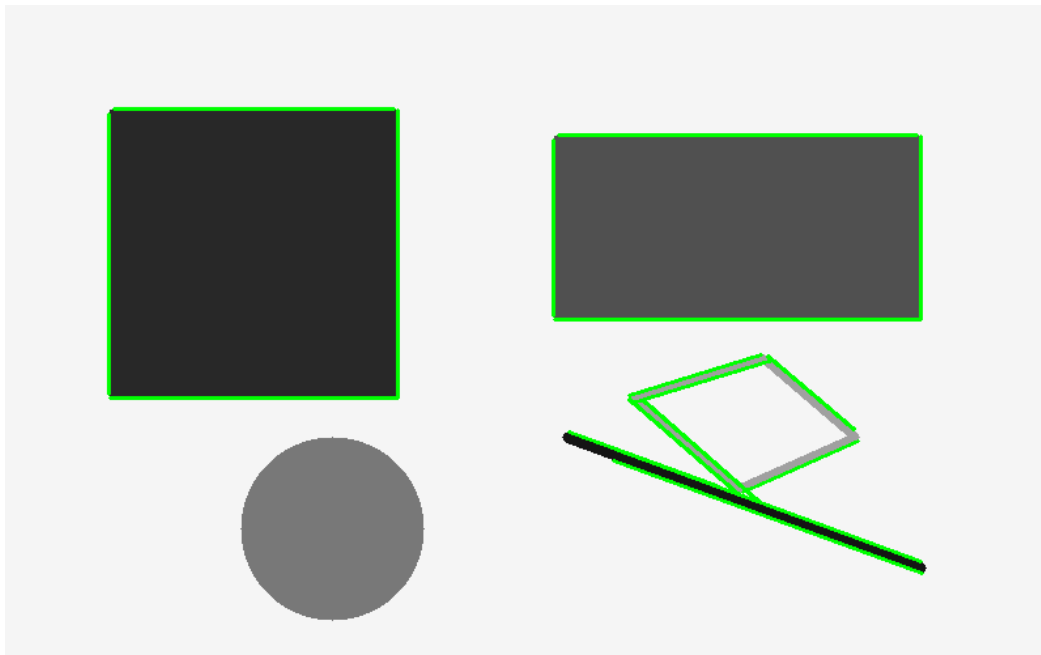


Figure 3. Hough line result (17 detected line segments).

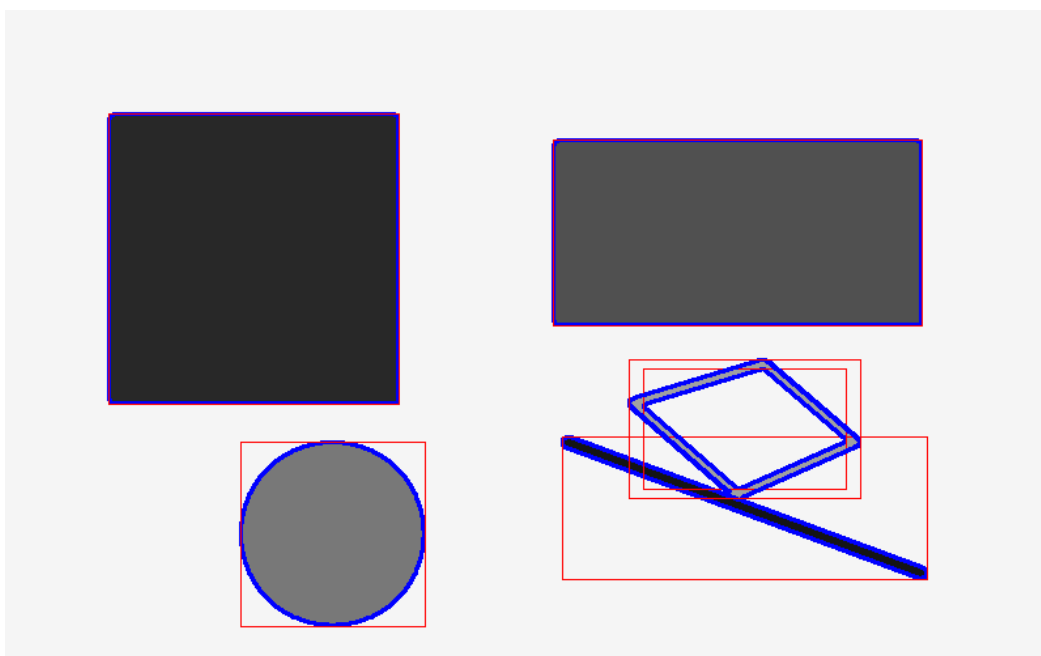


Figure 4. Contour analysis (6 contours above minimum area).

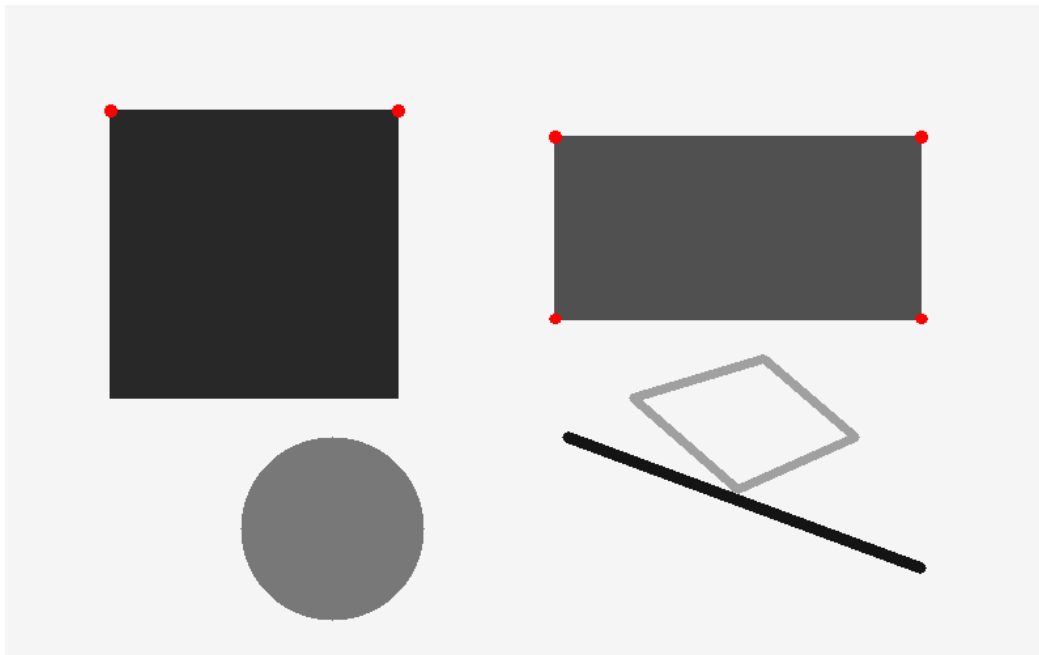


Figure 5. Harris corner result (80 displayed corner points).

11. Testing Approach

The project uses Python's unittest framework. Three automated tests verify that edge detection returns an image of the expected size, Hough line detection returns a valid image and count, and contour analysis returns a valid image and object list.

The end-to-end project was also executed using the generated sample image. The run completed successfully and produced all required result files. This confirms that the documented command-line workflow is executable.

Test command: `python -m unittest discover -s tests -v`

12. Challenges Faced

- Choosing useful default thresholds while keeping the system simple enough for an academic demonstration.
- Reducing false or insignificant contours by applying a minimum-area filter.
- Handling invalid image paths so the program fails clearly instead of silently producing empty output.
- Keeping the project modular while allowing all modules to work together through a simple CLI.
- Making the result reproducible without requiring an external image dataset.

13. Learnings & Key Takeaways

- Understood how image preprocessing affects later Computer Vision stages.
- Learned the practical role of Canny edge detection in boundary extraction.
- Understood how the Hough Transform can convert edge evidence into detected straight lines.
- Learned how contours can be used for object geometry such as area and bounding boxes.
- Understood the basic principle of Harris corner detection.
- Learned how to organize a Computer Vision program into reusable modules.
- Practiced command-line execution, validation, automated testing, and technical documentation.

14. Future Enhancements

- Add circle detection using HoughCircles.
- Add batch processing for complete folders of images.
- Add perspective correction for photographed documents or scenes.
- Add object classification using a machine-learning model as an optional extension.
- Add quantitative evaluation using labeled benchmark images and precision/recall metrics.
- Add configurable contour filtering and interactive threshold tuning while preserving the CLI.
- Add structured logging and export to CSV for batch analysis.

15. References

- OpenCV documentation – image processing and feature detection APIs.
- OpenCV documentation – Canny Edge Detection.
- OpenCV documentation – Hough Line Transform.
- OpenCV documentation – Contours and contour features.
- OpenCV documentation – Harris Corner Detector.
- Python documentation – argparse, pathlib, json, and unittest.